

Create Classes with Fields, Constructors, and Methods

Interview Goal: Understand how Java models real-world objects using classes. Interviewers usually ask this before moving to OOP concepts like encapsulation, inheritance, and polymorphism.

Learning Objectives

After this topic, you will be able to:

- Create a class.
 - Add fields (variables).
 - Create constructors.
 - Create methods.
 - Create objects from a class.
 - Understand the role of each component.
-

1. What is a Class?

A **class** is a blueprint for creating objects.

Example:

```
class Student {  
  
}
```

Think of a class as a template.

- Class = Blueprint
 - Object = Actual thing created from blueprint
-

2. Fields (Instance Variables)

Fields store the data of an object.

```
class Student {  
  
    String name;  
    int age;  
}
```

Here:

- name → field
 - age → field
-

3. Creating an Object

```
Student s1 = new Student();  
  
s1.name = "Ali";  
s1.age = 22;
```

4. Constructors

A constructor initializes an object.

Default Constructor

```
class Student {  
  
    Student() {  
        System.out.println("Object Created");  
    }  
}
```

Runs automatically when an object is created.

```
Student s1 = new Student();
```

Parameterized Constructor

```
class Student {  
  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Usage:

```
Student s1 = new Student("Ali", 22);
```

Interview Insight

Constructor vs Method

Constructor	Method
Same name as class	Any valid name
No return type	Has return type (or void)
Runs automatically	Called manually
Initializes object	Performs actions

5. Methods

Methods define behavior.

```
class Student {  
  
    String name;  
  
    void display() {  
        System.out.println(name);  
    }  
}
```

Usage:

```
Student s1 = new Student();  
  
s1.name = "Ali";  
  
s1.display();
```

Output:

```
Ali
```

Complete Example

```
class Student {  
  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void display() {
```

```
        System.out.println(name + " " + age);  
    }  
}
```

```
Student s1 = new Student("Ali", 22);  
  
s1.display();
```

Output:

```
Ali 22
```

Interviewer's Reasoning

When asked:

Why do we need constructors?

Don't just say:

"To create objects."

Better answer:

"Constructors ensure an object starts in a valid initialized state by assigning required values when it is created."

Common Mistakes

✗ Giving a constructor a return type

```
class Student {  
  
    void Student() {  
  
    }  
  
}
```

This is **not a constructor**.

It's a normal method.

 **Forgetting this**

```
Student(String name) {  
    name = name;  
}
```

Both refer to the parameter.

Correct:

```
Student(String name) {  
    this.name = name;  
}
```

Tricky Interview Questions

Q1. Can a class exist without a constructor?

Yes.

Java provides a **default constructor** if you don't create one.

Q2. Can a constructor return a value?

No.

Constructors never have a return type.

Q3. Can a class have multiple constructors?

Yes.

This is called **constructor overloading**.

```
Student() { }  
  
Student(String name) { }
```

Q4. Can a class have fields but no methods?

Yes.

Although it may not be very useful.

Output Question

Question

```
class Student {  
  
    String name;  
  
    Student() {  
        name = "Ali";  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student s = new Student();  
  
        System.out.println(s.name);  
    }  
}
```

```
}  
}
```

Output:

```
Ali
```

Cheat Sheet

Component	Purpose
Class	Blueprint
Object	Instance of class
Field	Stores data
Constructor	Initializes object
Method	Defines behavior
this	Refers to current object's field

Top Interview Questions

1. What is a class?
2. What is an object?
3. What is the purpose of a constructor?
4. Constructor vs Method?
5. Can constructors be overloaded?
6. Why do we use this?
7. What happens if no constructor is written?
8. Can a constructor have a return type?

Interview Tip

A common question is:

"What are the three basic building blocks inside a simple Java class?"

Answer:

Fields (state), Constructors (initialization), and Methods (behavior).

This is exactly how Java models real-world entities and is the foundation of Object-Oriented Programming.